Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Аппаратное и программное обеспечение сетей»

Тема: «Анализ сетевого трафика и протоколов на базе Wireshark»

Выполнил:
Студент 3 курса
Группы ИИ-28
Асташёнок Д.С.
Проверила:
Кулик А.Д.

Брест 2026

Цель работы

Изучить приёмы захвата и фильтрации трафика в Wireshark, получить статистику сеанса, разобрать заголовки Ethernet, IP, TCP, UDP, ICMP и ARP на реальных пакетах. Для прикладных сеансов использованы TCP- и UDP-серверы из лабораторных работ 1 и 2 (вариант 2.7: команда save … #end##).

Условия захвата

Захват выполнен анализатором tshark/dumpcap 3.6.2 в формате pcapng. Страница bstu.by открывалась командой curl; утилита ping — к шлюзу и к 1.1.1.1; TCP/UDP — к серверам на 127.0.0.1:8080 и 127.0.0.1:6666. Интерфейс any дал Linux cooked-capture (SLL) для внешнего трафика; интерфейс lo — Ethernet-кадры сеансов клиент–сервер.

Фильтры захвата и отображения

```
захват HTTP/HTTPS:   tcp port 80 or tcp port 443
отображение HTTP:    http
отображение IP:      ip
захват ping:         icmp or arp
отображение ping:    icmp
захват TCP-сеанса:   tcp port 8080
захват UDP-сеанса:   udp port 6666
захват ARP:          arp
```

3. Загрузка страницы bstu.by

Файл http-bstu.pcapng. Сайт отвечает редиректом HTTP 301 на HTTPS, поэтому в дампе есть и открытый HTTP, и TLSv1.3. Длительность 5,53 с, 1006 кадров, 1861 КБ полезных данных.

Статистика (Statistics → Protocol Hierarchy / IO Graph / Packet Lengths): 100 % кадров — IPv4/TCP. Из них HTTP — 4 кадра (0,4 % кадров, 1668 байт), TLS — 275 кадров (27,3 % кадров, ~1204 КБ). Остальное — служебный TCP (SYN/ACK, ACK, сегменты без разбора прикладного протокола). Средняя скорость 181 кадр/с и 336 КБ/с (2,691 Мбит/с). Размер кадра: min 56, max 26280, среднее 1850 байт. Если принять канал 100 Мбит/с, загрузка за интервал захвата ≈ 2,7 %.

Начало дампа (handshake HTTP, затем переход на 443):

```
1  172.17.0.2 → 93.125.24.147 TCP 45932→80 [SYN]
2  93.125.24.147 → 172.17.0.2 TCP 80→45932 [SYN,ACK]
3  ACK
4  HTTP GET / HTTP/1.1
6  HTTP/1.1 301 Moved Permanently
8  TCP 45150→443 [SYN]
11 TLSv1 Client Hello
```

4. Только IP-пакеты. Ethernet и IP

Фильтр отображения ip оставляет все 1006 кадров: в этом захвате нет кадров без IP. Доли прикладных протоколов стека TCP/IP те же (HTTP 0,4 % / TLS 27,3 % / прочий TCP). Размеры пакетов не изменились (min 56, max 26280, среднее 1850,16).

Структура Ethernet на примере SYN-кадра TCP-сеанса (файл tcp-session.pcapng, кадр 1). На loopback адреса обнулены, тип EtherType = 0x0800 (IPv4):

```
Ethernet II
  Destination MAC  00:00:00:00:00:00  (loopback)
  Source MAC       00:00:00:00:00:00
  Type             0x0800  IPv4
IPv4
```

```
Version          4
IHL              5  → 20 байт заголовка
DSCP/ECN         0 / Not-ECT
Total Length     60
Identification   0xd223
Flags            DF=1, MF=0, offset=0
TTL              64
Protocol         6 (TCP)
Src / Dst        127.0.0.1 → 127.0.0.1
```

Поля: Version/IHL задают IPv4 и длину заголовка без опций; Total Length — весь IP-пакет; DF запрещает фрагментацию; TTL ограничивает время жизни; Protocol выбирает TCP. Для внешнего HTTP-захвата вместо Ethernet использовался SLL (Linux cooked): тип канала Ethernet, протокол 0x0800, MAC источника интерфейса контейнера.

5. Утилита ping

Файл ping.pcapng, фильтр icmp. 16 кадров ICMP, все по 100 байт на проводе. Две пары: 172.17.0.2↔172.17.0.1 (шлюз, RTT ~0,05 мс, TTL 64 в обе стороны) и 172.17.0.2↔1.1.1.1 (RTT ~41–60 мс, ответ приходит с TTL 63). Потерь нет. Протоколы ping: ICMP (RFC 792) поверх IPv4; ARP в этом файле не попал — сосед уже был в кэше.

Интерпретация: Type=8 Code=0 — Echo Request; Type=0 Code=0 — Echo Reply. Identifier и Sequence связывают запрос и ответ (id=1 seq=1…4 к шлюзу, id=2 к 1.1.1.1). Контрольная сумма ICMP корректна. Данные 48 байт эха совпадают в запросе и ответе — канал не искажает payload.

Диаграмма обмена ping (шлюз):

```
хост 172.17.0.2                    шлюз 172.17.0.1
        |  ICMP Echo Request seq=1          |
        |---------------------------------->|
        |  ICMP Echo Reply   seq=1          |
        |<----------------------------------|
        |           … seq=2,3,4 …           |
```

6. TCP-сервер и TCP-клиент

Файл tcp-session.pcapng, фильтр tcp.port == 8080. 20 кадров, 0,42 с. Клиент 38778, сервер 8080. Сеанс: приветствие Hello, Student!; эхо «privet TCP»; команда save capture-tcp.txt; две строки файла; маркер #end##; ответ OK: file capture-tcp.txt created; четырёхстороннее закрытие.

Диаграмма TCP:

```
клиент:38778                              сервер:8080
   SYN ---------------------------------->
   <-------------------------------- SYN, ACK
   ACK ---------------------------------->
   <--------------------------- PSH Hello, Student!
   PSH privet TCP ---------------------------->
   <---------------------------- PSH privet TCP
   PSH save capture-tcp.txt ------------------>
   PSH pervaya stroka ------------------------>
   PSH vtoraya stroka ------------------------>
   PSH #end## ------------------------------->
   <-------------------- PSH OK: file … created
   FIN, ACK ---------------------------------->
   <-------------------------------- FIN, ACK
   ACK ---------------------------------->
```

Заголовок TCP (кадр 1, SYN). Source Port 38778, Destination Port 8080 — эфемерный порт клиента и порт сервиса. Sequence Number raw 94664354, относительный 0. Acknowledgment Number 0, флаг ACK сброшен — это начало handshake. Flags = SYN. Window 65495. Data

Offset 40 байт (10 слов) из□за опций: MSS 65495, SACK permitted, Timestamp, Window scale 7. Urgent Pointer 0. Контрольная сумма есть, в дампе не проверялась анализатором. PSH на кадрах с текстом означает, что сегмент нужно сразу отдать приложению.

7. UDP-сервер и UDP-клиент

Файл udp-session.pcapng, фильтр udp.port == 6666. 7 датаграмм, 5,18 с. Клиент 44010, сервер 6666. Эхо «privet UDP» имеет ответ; save и строки файла идут без ACK (UDP не гарантирует доставку, сервер по заданию молчит до #end##); после #end## приходит OK: file capture-udp.txt created.

Заголовок UDP (кадр 1). Source Port 44010, Destination Port 6666, Length 18 = 8 байт заголовка + 10 байт полезной нагрузки «privet UDP», Checksum 0xfe25. В IP при этом Protocol = 17, Total Length 38, TTL 64, DF=1. В отличие от TCP нет handshake, флагов и нумерации сегментов: каждая датаграмма самодостаточна.

8. ARP

Файл arp.pcapng. Перед ping шлюза хост 172.17.0.2 спрашивает MAC 172.17.0.1. Кадры 1–2: ARP Request, Opcode=1, Sender MAC 22:6e:cd:11:ca:8f, Sender IP 172.17.0.2, Target IP 172.17.0.1, Target MAC пустой. Кадр 3: ARP Reply, Opcode=2, 172.17.0.1 is at aa:68:5d:e3:ba:f5. Hardware type Ethernet (1), Protocol type IPv4 (0x0800), длины 6 и 4. После ответа IP-пакеты ping идут на известный MAC, ARP больше не нужен, пока запись жива в таблице соседей.

Вывод

Освоены фильтры захвата и отображения Wireshark/tshark, сбор статистики (иерархия протоколов, скорость, длины). На пакетах разобраны Ethernet/SLL, IPv4, TCP (handshake, PSH-данные, FIN), UDP и ICMP echo, а также ARP request/reply. Сеансы Java TCP/UDP варианта 2.7 видны в дампе полностью, включая save/#end##.